

优化与深度学习

目标

- 优化：减少训练误差
- 深度学习：减少泛化误差

风险

- 概念风险：整个数据分布的误差
 - 数据分布未知，无法计算
 - $E[\text{损失函数}(\text{模型预测}, \text{真实值})]$
- 经验风险：目前样本的误差
 - 可计算，也只能对其优化
 - $1/N \times \sum \text{损失函数}(\text{模型预测}_i, \text{真实值}_i)$

挑战

- 局部最小值
- 鞍点
 - 所有梯度消失
 - 但既不是局部最小值，也不是全局最小值
- 梯度消失
 - 可能会导致优化停滞
 - 可以用好的初始化来减少风险

梯度下降

一维

使用公式来迭代 x ，其中 η 为学习率

$$x \leftarrow x - \eta f'(x)$$

多维

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x}).$$

牛顿法

- 不仅关注一阶导，还关注二阶导
- 但计算量太大，往往不用

随机梯度下降 SGD

随机梯度更新

为了降低计算量，可以在每次迭代中，对数据样本随机均匀采样一个索引 i ，计算梯度并更新 $\mathbf{x}$

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f_i(\mathbf{x}),$$

但由于该思路引入了随机性，故难以收敛到具体的值，因此需要动态调整学习率

动态学习率

随着时间流逝，学习率也应当有变化，以下为主要思路

- 分段常数
 - `ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 9: \eta(t) \&= \eta_i \text{...}`
- 指数衰减
 - `ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 9: \eta(t) \&= \eta_0 \cdot \text{...}`
 - 往往会导致过早停止
- 多项式衰减
 - `ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 9: \eta(t) \&= \eta_0 \cdot \text{...}`

小批量随机梯度下降 Mini-batch SGD

SGD的统计效率和大批量一次处理数据的计算效率存在权衡，mini-batch SGD则可以双赢

- 使用小批量的样本来更新
- 既不是仅有1个，也不是全都用

不同改进算法

动量法

泄露平均值

即让旧数据随时间漏掉，表现为数据越旧，则权重越小

动量法

核心思想：给梯度下降加一个惯性，累计历史梯度，可以抵消震荡。其中 $\mathbf{v}_t$ 是速度， β 指之前多少速度被保留

$$\begin{aligned}\mathbf{v}_t &\leftarrow \beta \mathbf{v}_{t-1} + \mathbf{g}_{t,t-1}, \\ \mathbf{x}_t &\leftarrow \mathbf{x}_{t-1} - \eta_t \mathbf{v}_t.\end{aligned}$$

AdaGrad

- 问题：学习率对于不同频率的特征，其降低效果不同
- 解决方法：使用粗略的计数器来调整学习率，使用 s_t 来累加过去梯度 g_t 方差，让频繁出现的特征学习率自动变小
 - 对应较大梯度的坐标显著减小
 - 其他梯度较小的坐标则平滑处理

$$\begin{aligned} \mathbf{g}_t &= \partial_{\mathbf{w}} l(y_t, f(\mathbf{x}_t, \mathbf{w})), \\ \mathbf{s}_t &= \mathbf{s}_{t-1} + \mathbf{g}_t^2, \\ \mathbf{w}_t &= \mathbf{w}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \cdot \mathbf{g}_t. \end{aligned}$$

特点

- 不用调lr，会在单个坐标层面动态降低lr
- 利用梯度大小来调整进度速率，用较小的lr补偿带有较大梯度的坐标
- 在部分情况下，由于缺乏规范化，lr会衰减到0

RMSProp

在Adagrad中， $\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2$ ， s_t 会持续增长，最终学习率会趋于0。因此，可以调整 s_t 的计算公式：

$$\begin{aligned} \mathbf{s}_t &\leftarrow \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^2, \\ \mathbf{x}_t &\leftarrow \mathbf{x}_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t. \end{aligned}$$

其中，规范化体现在：

$$\begin{aligned} \mathbf{s}_t &= (1 - \gamma) \mathbf{g}_t^2 + \gamma \mathbf{s}_{t-1} \\ &= (1 - \gamma) (\mathbf{g}_t^2 + \gamma \mathbf{g}_{t-1}^2 + \gamma^2 \mathbf{g}_{t-2}^2 + \dots). \end{aligned}$$

由于 $1 + \gamma + \gamma^2 + \dots = \frac{1}{1-\gamma}$ ，所以权重综合标准化为1

特点

- RMSprop与Adagrad算法相似，都用梯度平方来缩放系数
- RMSprop与动量法都泄露平均值

Adadelta

- 是AdaGrad的变体，减少了学习率适应坐标的数量，可以完全自动确定学习率，**不常用**

- 可以使用参数本身变化率来自动调整学习率
- 需要两个状态变量来存储梯度二阶导和参数变化

Adam

集大成者，同时使用动量和自适应学习率，相当于动量法+RMSProp

过去的技术：

- **SGD** 比 **梯度下降** 更有效
- **Mini-batch SGD** 相比于SGD更高效
- **动量法** 可以汇总过去梯度历史，以加速收敛
- **AdaGrad** 通过对每个坐标缩放，以实现高效计算
- **RMSProp** 调整学习率来分离每个坐标的缩放

关键组成部分之一：

- 使用指数加权移动平均值
- 以此估算梯度的动量和二次矩

$$\begin{aligned}\mathbf{v}_t &\leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{s}_t &\leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2.\end{aligned}$$

其中，

- g_t, v_t, s_t 表示梯度、其一阶矩和二阶矩
- β_1, β_2 为对应的衰减率，常为0.9和0.999

通常，若初始化 $v_0 = s_0 = 0$ ，则需要对v和s进行标准化，即：

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_1^t} \text{ and } \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}.$$

此时，可以重新缩放梯度

$$\mathbf{g}'_t = \frac{\eta \hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \epsilon}.$$

最后再更新参数

$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \mathbf{g}'_t.$$

步骤	动量法	RMSProp	Adam
动量（一阶矩）	$\mathbf{v}_t \leftarrow \beta \mathbf{v}_{t-1} + \mathbf{g}_{t,t-1}$	—	$\mathbf{v}_t \leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t$
自适应率（二阶矩）	—	$\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t^2$	$\mathbf{s}_t \leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$
偏差校正	—	—	$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_1^t}, \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}$
更新量	$\mathbf{g}'_t = \eta_t \mathbf{v}_t$	$\mathbf{g}'_t = \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t$	$\mathbf{g}'_t = \frac{\eta \hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t + \epsilon}}$
参数更新	$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \mathbf{g}'_t$	$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \mathbf{g}'_t$	$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \mathbf{g}'_t$

lr调度器

- 单因子衰减
 - 学习率逐步下降，平滑指数衰减
 - 避免下降太多
 - $\eta_{t+1} \leftarrow \max(\eta_{\min}, \eta_t \cdot \alpha)$
- 多因子衰减
 - 固定epoch骤降，阶梯状
- 余弦调度器
 - 学习率按余弦曲线平滑下降
 - $\eta_t = \eta_T + \frac{\eta_0 - \eta_T}{2} (1 + \cos(\pi t / T))$
- 预热
 - 先从小学习率开始，逐步上升
 - 然后再从高学习率开始，平滑下降